Comparative Study of Collaborative Filtering Methods for Movie Recommendation

on MovieLens Latest 2023 Dataset with 33.8M ratings

Yuting Zhou · Yiwei Li

Introduction

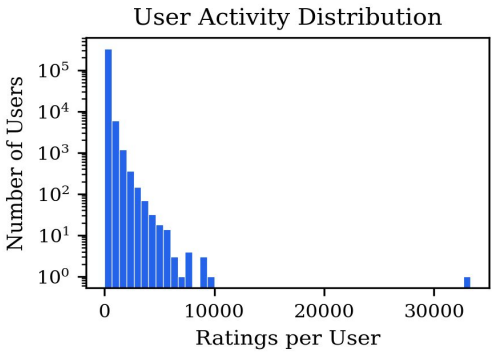
In this project, we implemented **SIX** distinct collaborative filtering models from scratch. All models are trained on the same MovieLens Latest split and evaluated using a unified metrics suite.

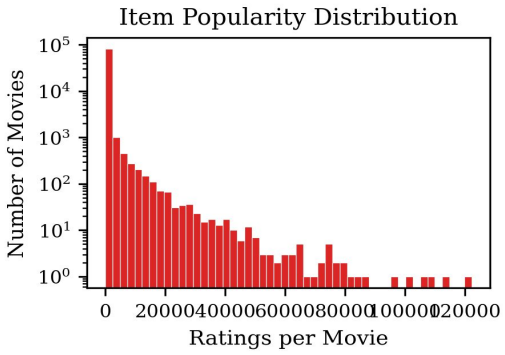
Our contributions are threefold:

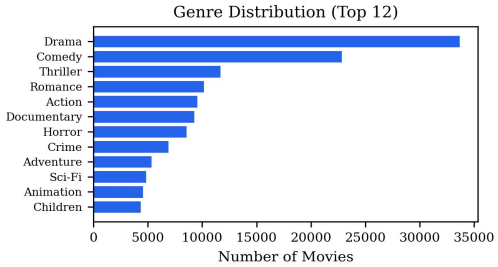
1. Provide a controlled comparison of CF models under identical data conditions:
   - ✓ MF-SGD
   - ✓ Deep Hybrid CNN
   - ✓ SVD-Ridge, SVD-KNN, SVD-Lasso
   - ✓ MF-ALS
2. Analyze not only aggregate accuracy (MAE,RMSE) and ranking quality (Precision, Recall, F1, NDCG@10) but also training dynamics, latent factor semantics, and feature block ablation.
3. Build ***StreamX***, an end-to-end web application (Django + Next.js) that serves real-time recommendations from the trained models, demonstrating practical deployment.

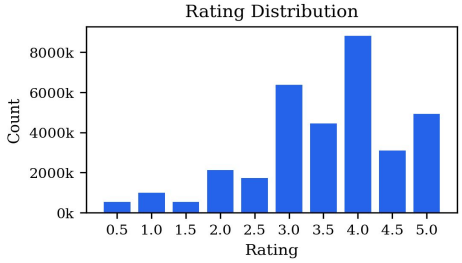
Dataset: MovieLens Latest 2023 Characteristics

Statistic	Value
Users	330,975
Rated movies	83,239
Ratings	33,832,162
Density	0.12%
Sparsity	99.88%
Mean rating	3.54
Median ratings/user	31.0
Median ratings/item	5.0









Heavy-Tailed Users (Gini ≈ 0.70)

Majority rate <100 items; power users exceed 10K. Regularization must account for varying observation counts across users.

Per-User 80/20 Split

Each user independently splits; ensures all models see the same user history, making comparisons directly valid.

Long-Tail Items (Gini ≈ 0.95)

Most movies receive <10 ratings — cold-start is endemic. Content features (title, genre) help denoise tail items.

Methodology: Six Collaborative Filtering Models

All models follow a common prediction form:

$$\hat{r}_{ui} = \mu + b_u + b_i + p_u^\top q_i$$

We evaluate them under the same protocol using a fixed-seed per-user 80/20 holdout split, resulting in *Dtrain* = 27.07M and *Dtest* = 6.77M ratings.

Model	Type	k	Loss	Optimizer	Epochs	Key Feature
MF-SGD	MF	64	MSE	Mini-batch SGD	≤150	LR decay 0.99×/ep; early stop
Deep Hybrid	Neural CF	96	Huber	Adam + plateau	50	CNN title + genre embed
SVD-Ridge	SVD + Linear	48	MSE	Ridge (closed)	1	Calibration: $w^T\varphi(u,i)$
SVD-Lasso	SVD + Linear	48	MSE	Lasso (CD)	1	L1 sparsifies weights
SVD-KNN	SVD + Cosine	48	—	None	1	L2-norm item factors
MF-ALS	MF	96	MSE	Alternating LS	≤40	Block coordinate descent

Experimental Setup

Evaluation Protocols

All models share an identical per-user 80/20 holdout split (with fixed random seed):

- D_train: 27,065,494 ratings
- D_test: 6,766,668 ratings

Rating Prediction

MAE and RMSE computed over all 6.77M test ratings.

$$MAE = \frac{1}{|D_{test}|} \sum_{(u,i) \in D_{test}} |r_{ui} - \hat{r}_{ui}|$$

$$RMSE = \sqrt{\frac{1}{|D_{test}|} \sum_{(u,i) \in D_{test}} (r_{ui} - \hat{r}_{ui})^2}$$

Top-10 Recommendation

For each user, all unrated items are ranked by predicted score, and the top 10 are selected. All held-out test items treated as relevant.

Metrics:

MAE	Precision@10	F1@10
RMSE	Recall@10	NDCG@10

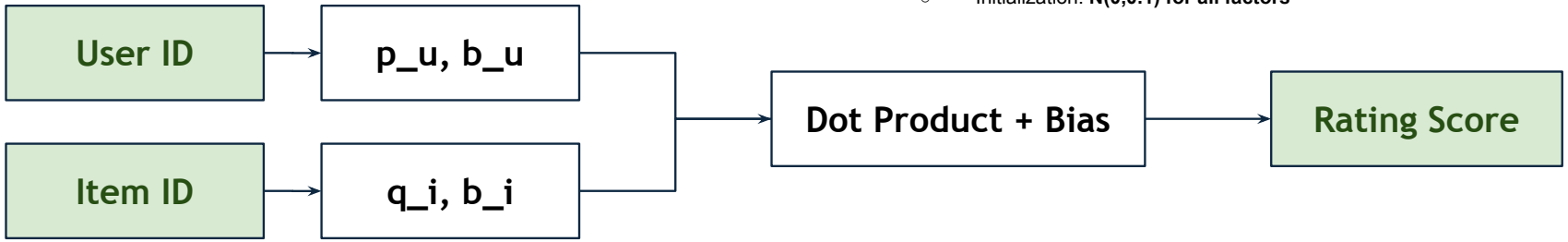
Matrix Factorization with SGD

L2-Regularized MSE Objective:

$$\min_{p_u, q_i, b_u, b_i} \sum_{(u,i)} (r_{ui} - \hat{r}_{ui})^2 \; + \; \lambda \left(\|p_u\|^2 + \|q_i\|^2 + b_u^2 + b_i^2 \right)$$

Hyperparameters

- Latent dimension: **k = 64**
- Learning rate: **η = 0.005, decay 0.99× / epoch**
- Regularization: **λ = 0.05 (L2)**
- Batch size: **131,072 ratings**
- Early stopping: **patience = 6 epochs**
- Initialization: **N(0,0.1) for all factors**



Design Motivation

Simple structure, interpretable, and low training cost; serves as a collaborative filtering baseline to assess the achievable upper bound using pure latent ID signals.

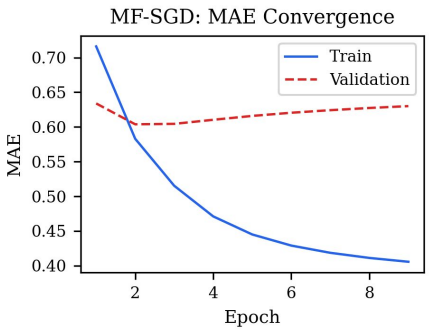
Risks ⚠

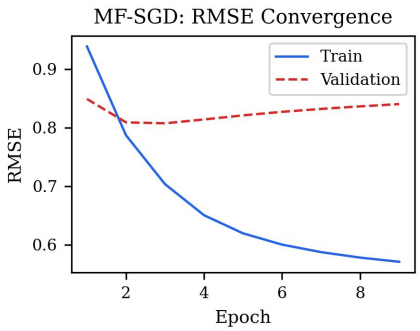
Optimizing only squared error can lead to early overfitting after reaching the best validation error; improvements in ranking quality are not guaranteed.

Engineering Considerations

Use large batch sizes to increase throughput, and apply validation checkpoint restoration to avoid post-training performance degradation.

MF-SGD ANALYSIS





0.6020	0.8061	0.0509	3
MAE	RMSE	NDCG@10	Best Epoch
2nd	BEST	3rd	

Strong RMSE, but Not Optimal for Ranking.

Strong Point-wise Accuracy

MF-SGD achieves the lowest RMSE among all methods, indicating that latent factors + bias terms effectively fit explicit ratings.

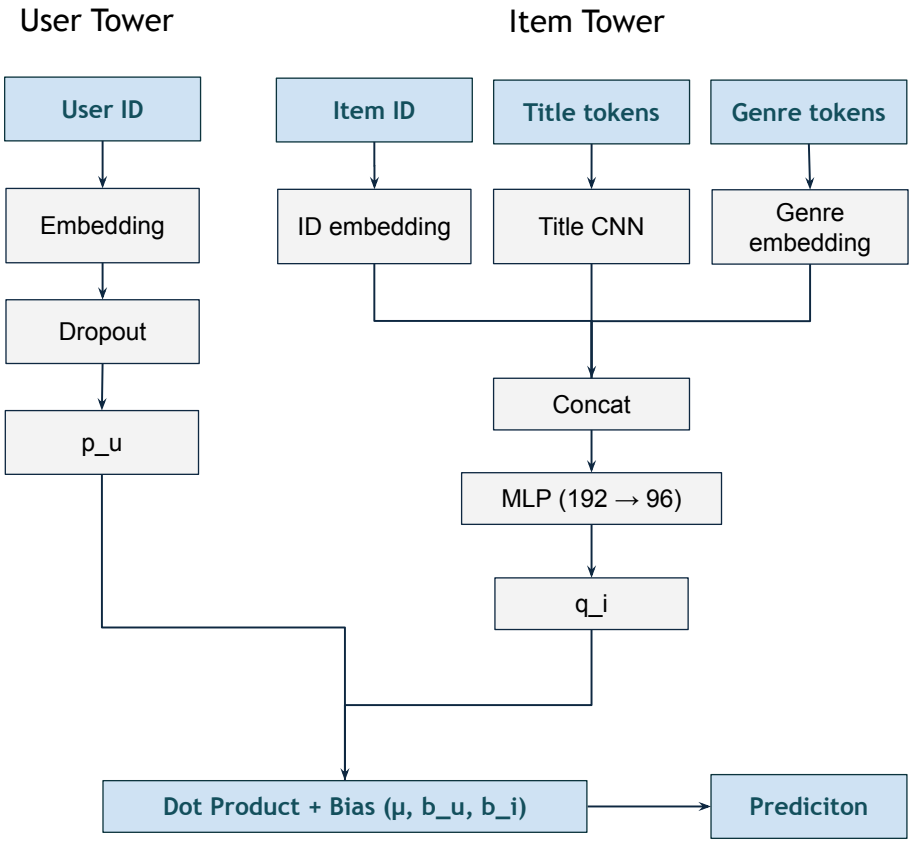
Weak Top-N Ranking ⚠

NDCG@10 is lower than Deep Hybrid and SVD-Ridge, suggesting that optimizing MSE does not directly separate top-10 candidates in ranking.

Takeaway

It is a strong baseline for rating prediction, but it highlights a key insight: rating accuracy and ranking quality are related but not equivalent objectives.

Deep Hybrid (Two-Tower + CNN)



Core Idea

Beyond collaborative filtering ID embeddings, introduce title CNN and genre pooling so the item tower can leverage content signals, alleviating long-tail sparsity.

Training Strategy

- Loss: Huber (δ = 1.0)
- Optimizer: Adam, LR 8×10^{-4}
- LR schedule: ReduceLROnPlateau (×0.5, patience 4)
- Gradient clip: L2 norm ≤ 1.0
- k = 96, dropout 0.18, L2 $\lambda = 10^{-5}$
- Batch: 65,536 (GPU); rating-weighted sampling

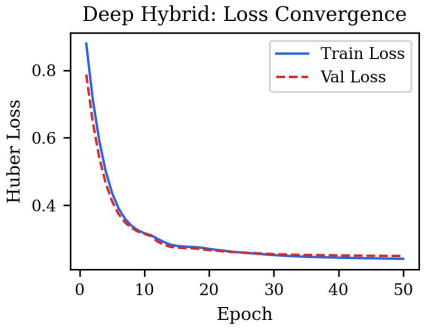
CNN Title Encoder

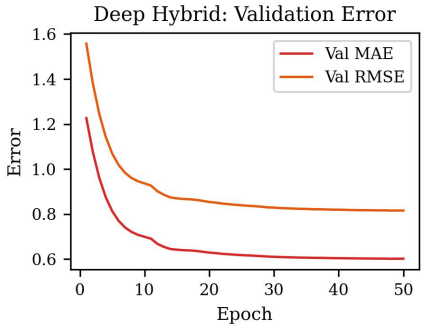
$$\mathrm{CNN}(t_i) = \mathrm{MaxPool}(\mathrm{ReLU}(\mathrm{Conv1D}_{k\in\{2,3,4\}}(E_t))) \in \mathbb{R}^{96}$$

- Genre: embed (d=8) → AvgPool → $\mathbb{R}^8$.
- Concatenated with ID embed → MLP → q_i

$$\hat{r}_{ui} = p_u^\top q_i + \mu + b_u + b_i$$

Deep Hybrid ANALYSIS





0.6005	0.0533	0.0480	0.0722
MAE	P@10	R@10	NDCG@10
BEST	BEST	BEST	BEST

Takeaway

If the goal is overall recommendation quality, Deep Hybrid is the primary model. For cost-sensitive deployment, it should be weighed against SVD (Ridge).

Why It Achieves the Best Overall Performance?

Mechanism Explanation

ID embeddings capture collaborative filtering signals, while the title CNN and genre pooling model longer-range item semantics, providing complementary content features. The Huber loss further stabilizes rating prediction.

Source of Ranking Gains

Rating-weighted sampling and content features make high-scoring items more distinguishable in ranking, leading to a clear improvement in NDCG@10.

Trade-off ⚠

The model is structurally more complex and relies more on GPU training and feature processing. However, at this data scale, the additional capacity translates into real performance gains.

SVD Representation + Three Heads

Shared Backbone

Construct a centered interaction matrix, apply truncated SVD to obtain user/item latent factors, then estimate bias shrinkage.

Rating matrix → Truncated SVD k = 48 → Bias shrinkage lambda_b → Feature phi(u,i) p_u * q_i + biases

Ridge
(L2 calibration, α = 0.1)

Strong linear head with stable ranking performance

Lasso
(L1 calibration, max_iter = 2000)

Sparse solution, but weaker ranking performance

KNN-style
(mean-normalized item vectors, cosine-like score)

Highly interpretable, but lacks proper calibration

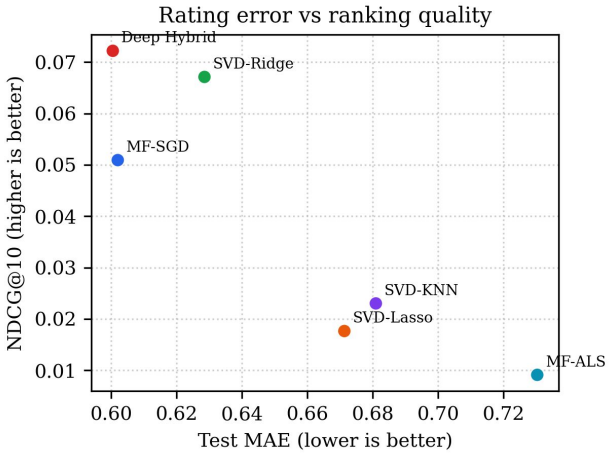
Design Positioning

SVD captures collaborative structure, while the head determines scoring calibration or geometric interpretation.

SVD ANALYSIS

Ridge Approaches Deep, Lasso/KNN Expose Head Differences

Head	MAE	RMSE	P@10	NDCG	Interpretation
Ridge	0.6286	0.8312	0.0520	0.0671	**Strong**
Lasso	0.6714	0.8747	0.0152	0.0177	**Too sparse**
KNN	0.6810	0.9058	0.0159	0.0230	**Uncalibrated**



Why Ridge Works?

L2 calibration re-adjusts the SVD latent product and bias terms, suppressing SVD noise while preserving sufficient score diversity.

Why Lasso Underperforms?

L1 sparsity reduces calibration weight diversity, leading to acceptable rating error but significantly degraded top-N ranking.

Why KNN is Meaningful?

User vectors represent the centroid of rated item factors, essentially a latent-space average. It is highly interpretable, but lacks regression-based calibration.

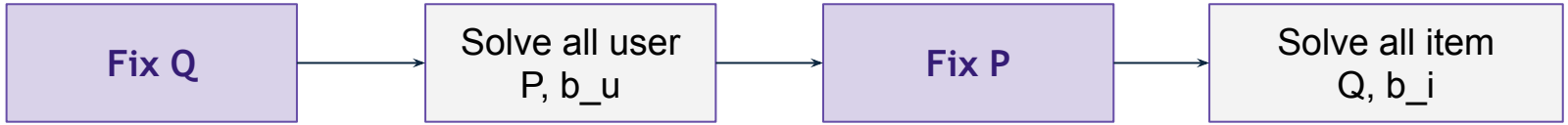
Alternating Least Squares

Using Ridge Regression while keeping user embeddings fixed:

$$\tilde{p}_u = (Q_u^\top Q_u + \Lambda)^{-1} Q_u^\top \tilde{y}_u$$

Hyperparameters

k=96, reg =0.16, bias_reg =6.0, epochs =40, validation split =10%, early stopping patience = 6



Core Idea

Fix item factors Q, solve a regularized least-squares problem for each user; then fix user factors P and solve the symmetric problem for each item, alternating until convergence.

Theoretical Advantages

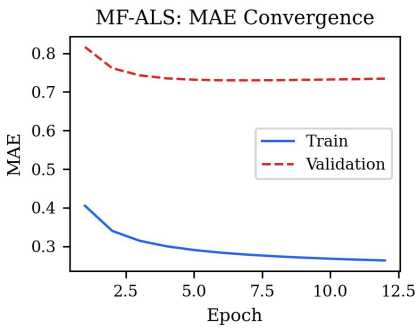
Each block update is a closed-form least-squares solution; well-suited for sparse matrices and a strong scalable baseline in classical collaborative filtering.

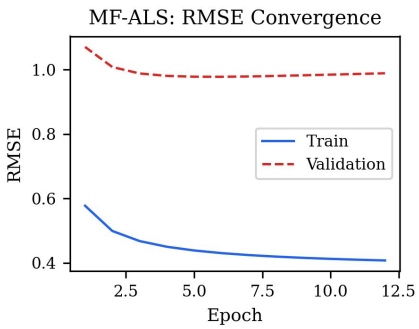
Challenges

Ranking performance is sensitive to the interaction of rank, regularization, and bias shrinkage under a long-tail sparse structure; current preset does not achieve strong ranking quality.

Note: This does not reject ALS; it indicates that, under explicit-feedback settings, a more systematic sweep over rank and regularization is still needed.

MF-ALS ANALYSIS





0.7303	0.9786	0.0072	0.0091
MAE	P@10	NDCG@10	NDCG@10
WORST	WORST	WORST	WORST

Takeaway

ALS is a meaningful but under-optimized classical baseline.

Its limitations highlight that evaluation cannot rely solely on training loss.

Training Objective Improves, but Validation and Ranking Lag Behind.

Overfitting / Mismatch ⚠

The training objective continues to decrease, but validation error plateaus early, indicating a mismatch between current regularization/ranking setup and the large-scale data split.

Weak Score Separation ⚠

Top-10 metrics are the lowest among all models, suggesting that predicted scores fail to effectively distinguish relevant from irrelevant items.

Next Steps

- ➔ Rank-aware hyperparameter sweep
- ➔ Per-block regularization
- ➔ Implicit-feedback ALS or confidence-weighted formulation

Results: Rating Prediction

Rating Prediction Performance Test

(Lower MAE and RMSE indicate better accuracy)

Key Findings (MovieLens Latest, 80/20 split)

- Deep Hybrid achieves most stable average prediction accuracy
- MF-SGD obtains the best RMSE, better control on large errors
- SVD-based methods show consistent performance gap → limited expressiveness
- MF-ALS performs worst → potential optimization or scalability issues

Takeaway

➔ Nonlinear modeling (Deep Hybrid) and well-optimized MF (SGD) significantly outperform classical CF variants.

➔ Traditional SVD-style extensions struggle at large scale and high sparsity.

Model	MAE↓	RMSE↓
Deep Hybrid	**0.6005**	0.8148
MF-SGD	0.6020	**0.8061**
SVD-Ridge	0.6286	0.8312
SVD-KNN	0.6810	0.9058
SVD-Lasso	0.6714	0.8747
MF-ALS	0.7303	0.9786

Table 3: Rating prediction on MovieLens Latest (per-user 80/20 test split). Lower is better.

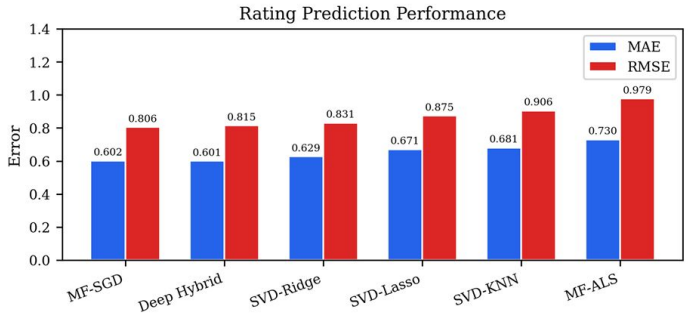


Figure 4: Side-by-side MAE and RMSE across all models (MovieLens Latest).

Results: Top-10 Recommendation

Top-10 Ranking Metrics (Higher is Better)

Holdout-style evaluation: all held-out test items per user treated as relevant.

Model	P@10↑	R@10↑	F1@10↑	NDCG@10↑
Deep Hybrid	**0.0533**	**0.0480**	**0.0331**	**0.0722**
SVD-Ridge	0.0520	0.0240	0.0212	0.0671
MF-SGD	0.0367	0.0294	0.0218	0.0509
SVD-KNN	0.0159	0.0200	0.0127	0.0230
SVD-Lasso	0.0152	0.0099	0.0063	0.0177
MF-ALS	0.0072	0.0038	0.0034	0.0091

⚠ Absolute values are small because each user has many held-out test items (all_test relevance definition). Relative ordering across models is the meaningful signal.

Interpretation

- Model ranking ability ≠ rating accuracy: MF-SGD is strong in RMSE but weaker in top-K ranking
- Nonlinear hybrid modeling (Deep Hybrid) better captures relative preference signals
- Absolute metric values are low due to all test items treated as relevant, so relative ordering is the key signal

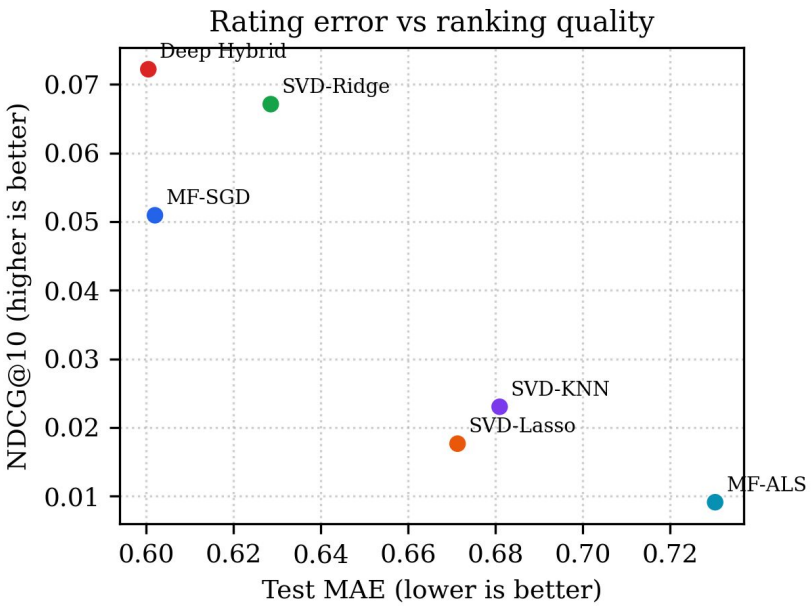
Takeaway

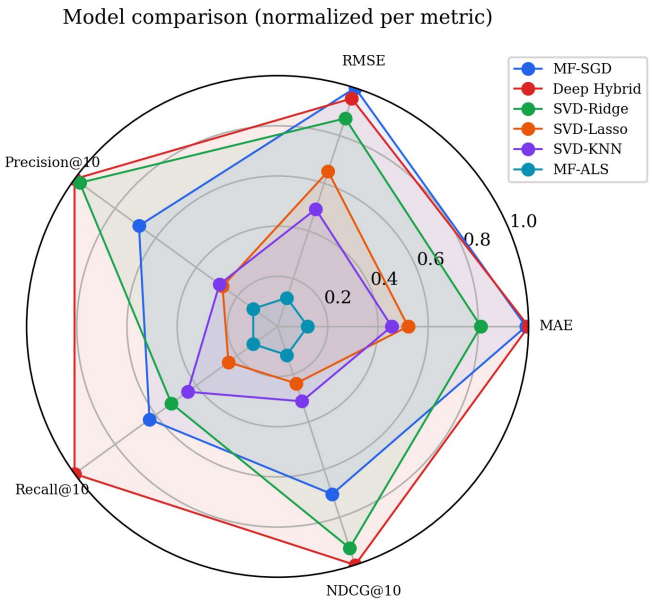
→ For recommendation tasks, optimizing ranking objectives is more aligned with UX than pure rating prediction

→ Deep Hybrid provides the best balance between representation power and ranking effectiveness

Multi-Metric Diagnosis

Model Roles Matter More Than a Single Ranking.





Primary Model

Deep Hybrid achieves the best overall performance, making it suitable for scenarios that demand high recommendation quality.

Lightweight Baseline

SVD-Ridge delivers ranking performance close to Deep Hybrid, with significantly lower training and interpretation cost.

Needs Improvement

ALS, Lasso, and KNN share common limitations in score calibration and ranking separation.

Training Dynamics

MF-SGD

Behavior

Rapid initial progress on training objective. Validation MAE/RMSE reverse direction within first few epochs.

Best Checkpoint

Typically epoch 2–4 despite training up to 150 epochs. Early stopping (patience 6) is essential.

Train error ↓ continuously; val error ↑ after epoch 4 → checkpoint restoration critical.

Deep Hybrid

Behavior

Smooth convergence under Huber loss with GPU minibatches (batch=65,536). Validation metrics improve steadily.

Best Checkpoint

Epoch 48 of 50-epoch run. ReduceLROnPlateau triggers LR reductions, stabilizing training.

Content features (title CNN + genre) help denoise tail items, improving both RMSE and NDCG.

MF-ALS

Behavior

Training error decreases monotonically. Validation error plateaus then drifts — train/val divergence.

Best Checkpoint

Early stopping triggers at epoch 12; restores epoch-6 checkpoint (k=96, λ=0.16).

Current preset is a baseline, not a fundamental limit. Large-scale MF-ALS requires rank/reg sweep, preconditioning, and possibly implicit-feedback formulation.

Feature-Block Ablation & Latent Factor Analysis

Feature-Block Ablation (MovieLens-Small-1M, Linear Probe)

Auxiliary study to understand which feature blocks contribute most to rating prediction beyond collaborative filtering.

Feature Block	R^2_full	R^2_w/o	ΔR^2
Full model	0.1116	—	—
w/o Content	—	0.0582	0.0534
w/o Engagement	—	0.0815	0.0302
w/o Noise (control)	—	0.1117	-0.0001

Feature blocks: (1) Content: genre indicators, genre count, release year, interaction year. (2) Engagement: log user activity, log item popularity, log tag count. (3) Noise: two random Gaussian features (control).

Content features account for the largest marginal contribution (ΔR^2=0.053), motivating the Deep Hybrid's title CNN and genre encoder. Most variance remains latent-ID-driven.

Latent Factor Interpretation (MF-SGD)

PCA on Item Factors

PCA applied to item factor matrix Q (k=64, |I|≈83K rated items). Principal components correlate with genre indicators and release year.

Semantic Axes

On smaller MovieLens slices, PCA yields coherent axes: e.g., mainstream blockbusters vs. critically acclaimed dramas from collaborative signal alone.

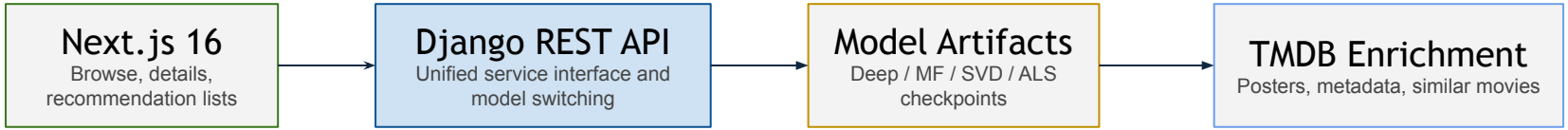
Full-Scale Caveats

At full scale, individual movie exemplars become noisier across training seeds. Interpretable directions are emphasized over selectively chosen examples.

Deployment: Full Stack Web Demo Architecture

StreamX

This demo translates offline experimental metrics into observable product behavior:
users can switch models and directly compare recommendation lists and similar-movie results.



Step	Input	API Action	Model Output	UI Result
1	User profile	Candidate scoring	Ranked movie IDs	Recommendations
2	Movie ID	Similarity lookup	Neighbor titles	Similar movies
3	Metadata	TMDB join	Poster + details	Detail page

Discussion & Conclusion

Limitations

- Uses a development dataset; metrics tied to July 2023 snapshot
- No automated hyperparameter search for MF-ALS and SVD-KNN
- Top-N evaluation on all_test leads to low absolute Precision/NDCG
- No modeling of temporal dynamics or implicit feedback

Future Work

→ Learning-to-rank optimization (e.g., BPR, NDCG-aligned objectives)
→ Large-scale rank and regularization sweep for MF-ALS
→ Rating-weighted KNN profile enhancement

StreamX Web Demo

- Built with Django + Next.js
- Supports real-time recommendations
- Integrated with TMDB metadata
- Enables runtime model switching
- Link: [StreamX: Movies Recommender]

Thank you

Comparative Study of Collaborative Filtering Methods
for Movie Recommendation

Yuting Zhou · Yiwei Li